Hierarchical Multi-Agent Reinforcement Learning for Congestion-Aware Maritime Scheduling

Project update: simulator background, reward redesign, controlled validation benchmark, current progress, and next steps.

SPRING 2026

Independent study

Supervisor: Prof. Amine Aboussalah

We now have both a strong continuous baseline and a controlled benchmark for validating the core control logic before reintroducing forecast uncertainty.

MARCH 31 UPDATE

Baseline: v3 + GT benchmark

RESEARCH FOCUS

HMARL

Coordinator, vessel, and port agents

CURRENT SETUP

v3 + GT

Operating baseline plus validation benchmark

FORMAT

12 slides

Presentation-ready update deck

Why this project matters

Operational context

- Ports have limited berth capacity and nonzero service time.
- Vessels trade off speed, fuel burn, emissions, and lateness.
- Poor local choices can create downstream congestion and idle capacity elsewhere.
- The system is naturally multi-agent, asynchronous, and resource-constrained.

Congestion

Too many arrivals to one port can create queueing and delay at the whole network level.

Cost

Faster sailing can improve timing but increases fuel use, emissions, and operating cost.

Coordination

Ports and vessels need global guidance, not only local reactive behavior.

Research value

This is a strong testbed for hierarchical coordination under delayed information.

What we are trying to learn

Umbrella question

Can hierarchical MARL improve maritime scheduling under congestion and resource constraints?

RQ1

How can heterogeneous agents coordinate using shared congestion forecasts?

RQ2

Does proactive coordination improve over independent and reactive baselines?

RQ3

Which forecast horizons and sharing strategies maximize decision quality?

RQ4

How do coordination improvements affect fuel, delay, and carbon cost?

The project is organized around coordination, baselines, forecast design, and economics.

Current simulator setup

Topology

5 ports, 8 vessels, 3 docks per port

Time scale

1 hour per step and 6 hour default service time

Travel

Synthetic port-distance matrix with routes from 22 to 98 nautical miles

Fuel

Default fuel is 100, with refueling only after real service completion

Synthetic port distance matrix (nautical miles)

Rows and columns are ports P0–P4; the diagonal is 0, and the matrix is symmetric.

The values are synthetic and scaled so multiple voyages can complete within one rollout.

	P0	P1	P2	P3	P4
P0	0	84	22	34	78
P1	84	0	98	61	54
P2	22	98	0	55	59
P3	34	61	55	0	82
P4	78	54	59	82	0

How the current baseline is trained and evaluated

Main operating baseline

- Algorithm: MAPPO with centralized training and decentralized execution
- Parameter sharing across vessels and across ports
- 100 training iterations and rollout_length = 64
- Environment horizon = 69
- Seed = 42
- Local CPU run for the reported baseline

Controlled validation benchmark

- single_mission episodes
- ground_truth forecasts
- Each vessel completes at most one trip per episode
- Used to validate control logic before reintroducing forecast error

Primary evaluation metrics

- On-time rate
- Completed arrivals
- Vessels served
- Dock utilization
- Total operating cost

Three decision layers with delayed coordination

Fleet coordinator

Strategic destination guidance Slowest decision cadence

→

Vessel agents

Speed control Arrival-slot requests

→

Port agents

Request acceptance Service and berth decisions

The environment is intentionally asynchronous, so the policies must coordinate under delayed information and physical travel time.

How one simulation step works

Transition kernel

1. Deliver messages

Apply due actions and delayed communications.

2. Advance vessels

Update motion, fuel burn, emissions, and arrivals.

3. Process ports

Update queue, berth service, service completion, and refueling.

4. Compute rewards

Log diagnostics and compute role-specific rewards.

Why this matters

- The simulator evolves through communication, travel, berth service, and reward calculation in sequence.
- That is why arrivals, delays, and service completions appear at different times in the logs.
- The next two slides separate reward intuition from reward formulas so the audience can follow both.

Reward structure in plain language

Vessel reward

Reward = arrival and punctuality minus travel cost

What it rewards

Arrival, On-time arrival

What it penalizes

Fuel used, Delay added, CO_2 emitted, Transit time, Schedule delay

Port reward

Reward = responsive berth management minus congestion cost

What it rewards

Accepted requests, Vessels served

What it penalizes

Rejected requests, Queue waiting, Idle docks

Coordinator reward

Reward = system flow and utilization minus network-wide cost

What it rewards

Accepted requests, Vessels served, Berth utilization

What it penalizes

Fuel, Queue, Idle docks, Delay, Schedule delay, CO_2, Rejected requests

This is the audience-facing summary. The next slide shows the actual symbolic formulas used in the simulator.

Actual reward formulas used in the simulator

Symbolic form

Vessel reward

$$r_V(t) = r_{arr} \cdot 1[\text{arrived}] + r_{on} \cdot 1[\text{on_time}] - (w_f \Delta\text{fuel} + w_d \Delta\text{delay} + w_e \Delta CO_2 + w_t \Delta\text{transit} + w_s \Delta\text{sched})$$

Port reward

$$r_P(t) = r_{acc} \cdot \text{accepted} + r_{srv} \cdot \text{served} - r_{rej} \cdot \text{rejected} - (\text{queue} \cdot dt + w_{idle} \cdot \text{idle_docks})$$

Coordinator reward

$$r_C(t) = r_{acc} \cdot \text{accepted} + r_{srv} \cdot \text{served} + r_{util} \cdot \text{avg_occupied} - (w_f \Delta\text{fuel} + w_q \text{avg_queue} + w_i \text{avg_idle} + w_d \Delta\text{delay} + w_s \Delta\text{sched} + w_e \Delta CO_2 + r_{rej} \cdot \text{rejected})$$

Δ terms are per-step increments; accepted, rejected, and served are event counts for the current step.

What changed since March 6

Corrected vessel behavior

No more collapse to constant minimum speed. Requested arrival times now become real schedule deadlines.

Fuel and stalling

Fuel exhaustion now causes true mid-route stalling. Departures are checked for fuel feasibility.

Port service realism

Ports track actual vessel IDs. Refueling happens after real service completion.

Reward redesign

Rewards moved from mostly aggregate penalties to event-driven, role-specific signals.

Validation variants

Added single_mission and ground_truth as controlled debugging modes.

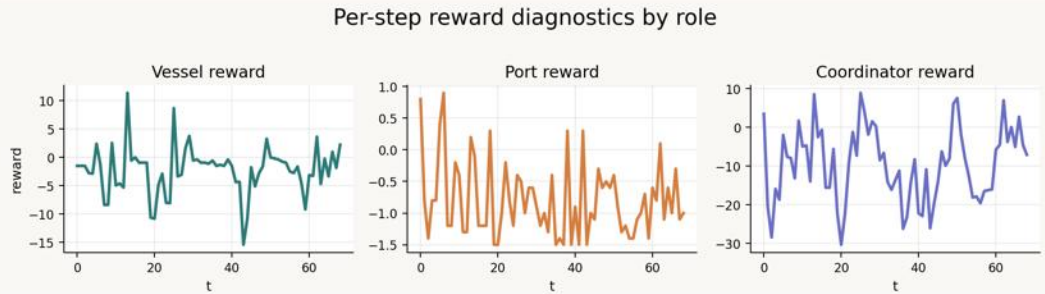
Visibility

Action logs, event logs, reward components, and confidence diagnostics were added.

Why the current results are more trustworthy

Transparency upgrades

- Grouped plots separate aggregate, vessel, port, and coordinator behavior.
- Per-step eval trace, action log, and event log are available.
- Reward-component decomposition shows which terms dominate.
- Policy-confidence metrics make near-uniform coordinator behavior easier to diagnose.



Role-specific reward traces over one evaluation episode. We show all three because vessel, port, and coordinator each optimize a different part of the task.

TRACE FILE
eval_trace.csv

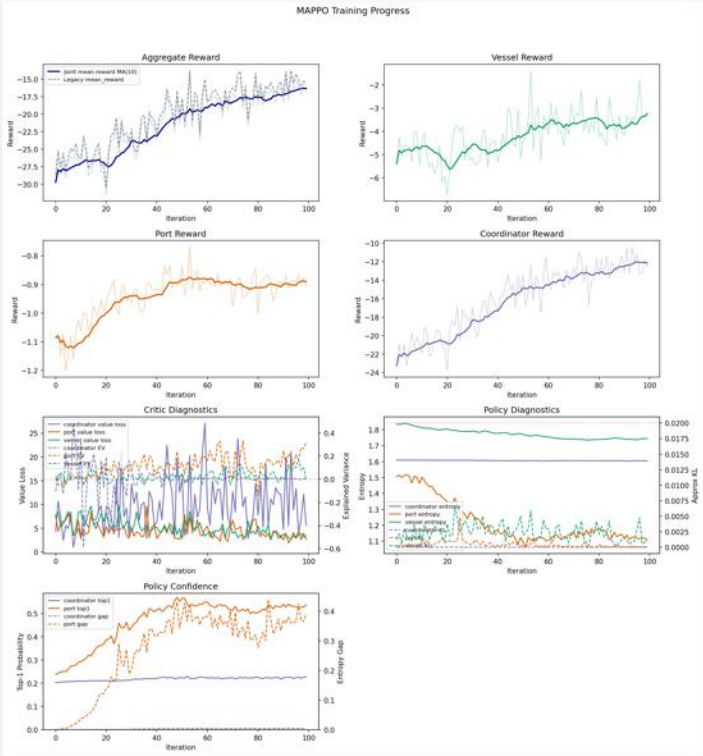
ACTION LOG
eval_action_trace.csv

EVENT LOG
eval_event_log.csv

Current recommended baseline: transit-rebalanced v3

Preferred operating balance

TOTAL REWARD **-923.38**	ON-TIME RATE **0.928**
COMPLETED ARRIVALS **26.8**	VESSELS SERVED **36.2**
DOCK UTILIZATION **0.28**	OPS COST **$1.330M**



Continuous task vs. single-mission validation benchmark

Continuous + ground truth

EVAL TOTAL REWARD	-673.99
ON-TIME RATE	0.759
COMPLETED ARRIVALS	14.8
VESSELS SERVED	24.2
MISSION SUCCESS	0.0
TRACE STEPS	37

Single mission + ground truth

EVAL TOTAL REWARD	-481.51
ON-TIME RATE	0.575
COMPLETED ARRIVALS	8.0
VESSELS SERVED	9.4
MISSION SUCCESS	1.0
TRACE STEPS	24

Takeaway: the simplified benchmark is easier to complete and gives a cleaner success signal, but the continuous setting still has higher throughput and remains the real target environment.

What is simplified, and what comes next

1. Keep v3

Use transit-rebalanced v3 as the working continuous baseline.

2. Use the benchmark

Use single_mission + ground_truth as the controlled validation benchmark.

3. Compare across seed

Run short multi-seed comparisons in both settings.

4. Reintroduce uncertai

Bring back imperfect forecasting after validating control behavior.

5. Decide on realism

Then decide whether to move to real ports and nautical distances.

The short-term goal is not to replace the continuous environment, but to validate the control logic under cleaner conditions.